

demo_classification

June 8, 2026

```
[ ]: """
Demonstration of my UniversalClassifier wrapper, based on Scikit-Learn and cuML
    ↪ and
    ↪ supporting GPU
    """

import warnings
import pandas as pd
from matplotlib import pyplot as plt
from ac.ml.classification import UniversalClassifier
from ac.tabular import Report

warnings.filterwarnings("ignore")
```

```
[ ]: """
Loading a scraped dataset containing data about the top 500 billionaires in the
    ↪ world: region, wealth and industry sector. The non-numerical columns have
    ↪ already been mapped.
    """

billionaires = pd.read_csv(".data/billionaires.csv")
billionaires.info()
```

```
[ ]: """
Let's define a "sample", the 501st richest man in the world, and let's see
    ↪ which algorithm
    ↪ predicts his country the best
    """

algorithms = ["decisiontree", "randomforest", "svm"]
columns = ["Wealth", "Mapped_Region", "Mapped_Country", "Mapped_Sector"]
sample = [80, 1.0, 3.0]
```

```
[ ]: """
UniversalClassifier explores automatically the optimal hyperparameter
    ↪ configuration.
Let's define the hyperparameter range
    """

grids = {
    "decisiontree": {
```

```

        "max_depth": [1, 2, 4, 8, 16, 32],
        "min_samples_leaf": [1, 2, 3, 4, 5],
        "criterion": ["gini", "entropy", "log_loss"],
    },
    "randomforest": {
        "max_depth": [1, 2, 4, 8, 16, 32],
        "min_samples_leaf": [1, 2, 3, 4, 5],
        "criterion": ["gini", "entropy", "log_loss"],
    }
}

```

```

[ ]: """
Launching the classification task. This cell demonstrates also my Report class,
which allows the generation of simple reports from tabular data in different
↳document
formats.
"""
for algorithm in algorithms:
    uc = UniversalClassifier(method=algorithm, data=billionaires,
    ↳subset=columns, target="Mapped_Country")
    uc.split(ratio="30%")

    # Exploring different configurations with cross-validation
    param_grid = grids.get(algorithm, None)
    if param_grid is not None:
        uc.explore(param=param_grid, cv=5)

    # Fitting and performing cross-validation
    uc.fit(cv=5)
    predicted = uc.predict(sample)

    # Plotting the algorithm performance
    if param_grid is not None:
        uc.draw("performance", figsize=(10, 8), rotation=90)
        plt.show()

    # Generating some textual reports
    results = uc.evaluate("scores")
    title = "Scores of " + algorithm
    report = Report()
    report.set(source=results, head="Scores")
    report.headline(title)
    report.textualize()

    if algorithm != "svm":
        results = uc.evaluate("params")
        title = "Parameter values of " + algorithm

```

```

report.set(source=results, head="Parameters")
report.headline(title)
report.textualize()

# Saving everything in pdf
report.dump(report_name=algorithm, extension=".pdf")

# Drawing the confusion matrix
uc.draw(what="confusion", set="test", title=algorithm, show=True)

# Dendrogram
if algorithm == "decisiontree":
    uc.draw(
        what="tree",
        title="Dendrogram of " + algorithm,
        max_depth=2,
        figsize=(24, 16),
        show=True
    )

```